Министерство науки и высшего образования РФ
федеральное государственное автономное
образовательное учреждение высшего образования
«Омский государственный технический университет»

Факультет информационных технологий и компьютерных систем

Кафедра «Прикладная математика и фундаментальная информатика»

Лабораторная работа №2

по дисциплине **Математические основы защиты информации**

Тема: Криптоанализ аффинного шифра

Студента	Бакланова Даниила Леонидовича
Курс	3 Группа МО-241
Направление	02.03.02 Фундаментальная информатика и информационные технологии
Руководитель	доц., канд. пед. наук, доцент
	Белим С.Ю.
Выполнил	29.10.2026
Проверил	29.10.2026

Омск 2025

Лабораторная работа № 2

Криптоанализ аффинного шифра

Вариант №1

Ф. И. О. студента: Бакланова Даниила Леонидовича

Группа: МО-241

Проверил: Белим Светлана Юрьевна Дата: 29.10.2026

Основные сведения

Формула для зашифрования текста: $y_i = E_k x_i = (ax_i + b) \; mod \; m$

Формула для расшифрования текста: $x_i = a^{-1}(y_i - b) \; mod \; m$

Результаты

ШИФР-ТЕКСТ (ШТ):

щковмтчгфюсякяфртичгфдесявжтрдясзруярдэсчфбефючхрэръвжмвгячхбрйебэчфчфгстнгюамвмякуежюутвщжнбеъкгфнчдщтндвсэчфкфмкукфмвежвэвссолкмгкжкфявруэеймчдфеэебчдмтвгяромеъвмч

Результаты частотного анализа ШТ:

Буква	а	б	в	г	д	е/ё	ж	з	и	й
частота	0.0061	0.0303	0.0788	0.045	0.0424	0.0606	0.0364	0.0061	0.0061	0.0121
Буква	к	л	м	н	о	п	р	с	т	у
частота	0.0606	0.0061	0.0667	0.0242	0.0182	0	0.545	0.0485	0.0424	0.0303
Буква	ф	х	ц	ч	ш	щ	ъ	ы	ь	э
частота	0.0788	0.0121	0	0.0727	0	0.0182	0.0182	0	0	0.0424
Буква	ю	я								
частота	0.0242	0.0545								

Наиболее часто встречающиеся символы: **В** (код 2)**, Ф** (код 20)

Системы уравнений: $\begin{cases} (a \cdot 0 + b) = 2 \\ (a \cdot 18 + b) = 20 \end{cases}$

Решения систем уравнений: (1,2), (17, 2)

ВЕРНЫЙ КЛЮЧ: a=17, b=2

РАСШИФРОВАННЫЙ ТЕКСТ (ОТ):

зимакрестьянинторжествуянадровняхобновляетпутьеголошадкаснегпочуплететсярысьюкакнибудьбраздыпушистыевзрываялетиткибиткаудалаяямщиксидитнаоблучкевтулупевкрасномкушаке

Код программы

```
from collections import Counter
import math
def extended_gcd(a, b):
    sa = 1 if a >= 0 else -1
    sb = 1 if b >= 0 else -1

    a, b = abs(a), abs(b)

    if a == 0:
        return b, 0, sb

    g, x1, y1 = extended_gcd(b % a, a)

    x = y1 - (b // a) * x1
    y = x1

    return g, x * sa, y * sb

def mod_inverse(a, m):
    a %= m
    g, x, _ = extended_gcd(a, m)

    return x % m if g==1 else None

def solve_congruence(a, b, m):

    a_mod = a % m
    b_mod = b % m

    g, _, _ = extended_gcd(a_mod, m)

    if b_mod % g != 0:
        return []

    a_prime = a_mod // g
    b_prime = b_mod // g
    m_prime = m // g

    inv_a = mod_inverse(a_prime, m_prime)

    x0 = (inv_a * b_prime) % m_prime

    solutions = []
    for k in range(g):
        sol = (x0 + k * m_prime) % m
        solutions.append(sol)

    return sorted(solutions)
def solve_system(a, b, c, d, m):
    A = (a - c) % m
```

```
    B = (b - d) % m

    x_solutions = solve_congruence(A, B, m)

    if not x_solutions:
        return []

    results = []
    for x in x_solutions:
        y = (b - a * x) % m
        results.append((x, y))

    return results
ALPHABET = "абвгдежзийклмнопрстуфхцчшщъыьэюя"
RAW_FREQ_DATA = {
    '-': 0.175,
    'о': 0.090,
    'е': 0.072,
    'ё': 0.072,
    'а': 0.062,
    'и': 0.062,
    'т': 0.053,
    'н': 0.053,
    'с': 0.045,
    'р': 0.040,
    'в': 0.038,
    'л': 0.035,
    'к': 0.028,
    'м': 0.026,
    'д': 0.025,
    'п': 0.023,
    'у': 0.021,
    'я': 0.018,
    'ы': 0.016,
    'з': 0.016,
    'ъ': 0.014,
    'ь': 0.014,
    'б': 0.014,
    'г': 0.013,
    'ч': 0.012,
    'й': 0.010,
    'х': 0.009,
    'ж': 0.007,
    'ю': 0.006,
    'ш': 0.006,
    'ц': 0.004,
    'щ': 0.003,
    'э': 0.003,
    'ф': 0.002
}
```

```
M = len(ALPHABET)
def get_top_letters(ciphertext):
    clean_text = ""
    for char in ciphertext.lower():
        if char == 'ë':
            char = 'e'
        if char in ALPHABET:
            clean_text += char

    counts = Counter(clean_text)
    total = sum(counts.values())

    sorted_chars = []
    for char, count in counts.most_common():
        freq = count / total if total > 0 else 0
        sorted_chars.append((char, freq))

    return sorted_chars

def codes_to_text(codes):
    return "".join(ALPHABET[c] for c in codes)

def calculate_score(plain_text):
    counts = Counter(plain_text.lower())
    total_len = sum(counts.values())

    if total_len == 0:
        return 0.0

    vec_actual = []
    vec_target = []

    for char in ALPHABET:
        actual_freq = counts.get(char, 0) / total_len
        vec_actual.append(actual_freq)

        target_freq = RAW_FREQ_DATA.get(char, 0.0)
        vec_target.append(target_freq)

    dot_product = sum(a * b for a, b in zip(vec_actual, vec_target))
    norm_a = math.sqrt(sum(a * a for a in vec_actual))
    norm_b = math.sqrt(sum(b * b for b in vec_target))

    if norm_a == 0 or norm_b == 0:
        return 0.0

    return dot_product / (norm_a * norm_b)

def decrypt_affine(ciphertext, a, b):
    inv_a = mod_inverse(a, M)
```

```
    if inv_a is None:
        return ""

    plain_codes = []
    for char in ciphertext.lower():
        if char == 'ё':
            char = 'е'

        if char in ALPHABET:
            y_code = ALPHABET.index(char)
            x_code = (inv_a * (y_code - b)) % M
            plain_codes.append(x_code)

    return codes_to_text(plain_codes)

LOG_FILE = "analysis_log.txt"

def write_log(message):
    with open(LOG_FILE, 'a', encoding='utf-8') as f:
        f.write(message + "\n")
        print(message)

def find_key_and_decrypt(ciphertext, threshold=0.9):
    cipher_stats = get_top_letters(ciphertext)

    c_char1, _ = cipher_stats[0]
    c_char2, _ = cipher_stats[1]

    code_c1 = ALPHABET.index(c_char1)
    code_c2 = ALPHABET.index(c_char2)

    write_log(f"\n--- НАЧАЛО АНАЛИЗА (ВЗВЕШЕННЫЙ ПЕРЕБОР) ---")
    write_log(f"Шифр-текст: {ciphertext}")
    write_log(f"Топ-2 буквы ШТ: '{c_char1}'({code_c1}), '{c_char2}'({code_c2})")

    candidates = [k for k in RAW_FREQ_DATA.keys() if k not in ('-', 'ё')]
    candidates.sort(key=lambda x: -RAW_FREQ_DATA[x])

    pairs = []
    for c1 in candidates:
        for c2 in candidates:
            if c1 == c2:
                continue
            w = RAW_FREQ_DATA[c1] * RAW_FREQ_DATA[c2]
            pairs.append((w, c1, c2))
    pairs.sort(key=lambda x: -x[0])

    best_score = -1
    best_result = None
    hyp_counter = 0
```

```
    for weight, p_char1, p_char2 in pairs:
        hyp_counter += 1

        temp_p1 = 'e' if p_char1 == 'ё' else p_char1
        temp_p2 = 'e' if p_char2 == 'ё' else p_char2

        if temp_p1 not in ALPHABET or temp_p2 not in ALPHABET:
            continue

        code_x1 = ALPHABET.index(temp_p1)
        code_x2 = ALPHABET.index(temp_p2)

        solutions = solve_system(code_x1, code_c1, code_x2, code_c2, M)

        if not solutions:
            continue

        for sol_a, sol_b in solutions:
            if extended_gcd(sol_a, M)[0] != 1:
                continue

            plaintext = decrypt_affine(ciphertext, sol_a, sol_b)
            score = calculate_score(plaintext)

            log_line = f"Нур #{hyp_counter}: ({p_char1},{p_char2}) w={weight:.5f}
Key:({sol_a},{sol_b}) Score:{score:.4f} Text:{plaintext[:40]}..."
            write_log(log_line)

            if score > best_score:
                best_score = score
                best_result = (sol_a, sol_b, plaintext)

            if score >= threshold:
                write_log(f"*** КЛЮЧ НАЙДЕН на Нур #{hyp_counter}
(Score={score:.4f}) ***")
                return (sol_a, sol_b, plaintext)

    if best_result:
        a, b, text = best_result
        write_log(f"ЛУЧШИЙ РЕЗУЛЬТАТ: Key=({a}, {b})\nText: {text}")
        return (a, b, text)
    else:
        write_log("Решений не найдено.")
        return None

def run_menu():
    while True:
        choice = input("\nВыберите пункт меню:\n 1. Проверка математики\n 2.
Криптоанализ\n 3. Выход\n-> ").strip()

        if choice == '1':
```

```
            sub_choice = input("\nПодпункт проверки:\n 1. Алгоритм Евклида\n 2.
Обратный элемент\n 3. Сравнение ax=b\n 4. Система уравнений\n-> ")

            if sub_choice == '1':
                try:
                    a = int(input("a = "))
                    b = int(input("b = "))
                    g, x, y = extended_gcd(a, b)
                    msg = f"[TEST EGCD] Input: a={a}, b={b} -> НОД={g}, x={x},
y={y}"
                    write_log(msg)
                except ValueError:
                    write_log("[ERROR] Invalid input")

            elif sub_choice == '2':
                try:
                    a = int(input("a = "))
                    m = int(input("m = "))
                    res = mod_inverse(a, m)
                    msg = f"[TEST INV] Input: a={a}, m={m} -> a^-1: {res}"
                    write_log(msg)
                except ValueError:
                    write_log("[ERROR] Invalid input")

            elif sub_choice == '3':
                try:
                    a = int(input("a = "))
                    b = int(input("b = "))
                    m = int(input("m = "))
                    sols = solve_congruence(a, b, m)
                    msg = f"[TEST CONGRUENCE] Input: a={a}, b={b}, m={m} -> x:
{sols}"
                    write_log(msg)
                except ValueError:
                    write_log("[ERROR] Invalid input")

            elif sub_choice == '4':
                try:
                    a = int(input("a = "))
                    b = int(input("b = "))
                    c = int(input("c = "))
                    d = int(input("d = "))
                    m = int(input("m = "))
                    sols = solve_system(a, b, c, d, m)
                    msg = f"[TEST SYSTEM] Input: sys({a},{b};{c},{d}) mod {m} ->
Solutions: {sols}"
                    write_log(msg)
                except ValueError:
                    write_log("[ERROR] Invalid input")
            else:
                print("Неверный выбор подпункта.")
```

```
        elif choice == '2':
            crypto_sub = input("\nРежим криптоанализа:\n 1. Вариант 1 (готовый
текст)\n 2. Частотный анализ (произвольный текст)\n 3. Перебор ключей
(произвольный шифр-текст)\n-> ")

            if crypto_sub == '1':
                ciph_var1 =
"щковмтчгфюсякяфртичгфдесявжтрдясзруярдэсчфбефючхрэръвжмвгячхбрйебэчфчфгстнгюамвм
якуежюутвщжнбеькгфнчдщтндвсэчфкфмкукфмвежвэвссолкмгкжкфявруэеймчдфеэебчдмтвгяроме
ъвмч"
                result = find_key_and_decrypt(ciph_var1)
                if result:
                    a_found, b_found, text_found = result
                    write_log(f"\nИТОГ ВАРИАНТА 1:")
                    write_log(f"КЛЮЧ: a={a_found}, b={b_found}")
                    write_log(f"ТЕКСТ: {text_found}")
            elif crypto_sub == '2':
                text = input("Введите текст для анализа частот: ").strip()
                if not text:
                    print("Текст пуст.")
                else:
                    stats = get_top_letters(text)
                    write_log(f"\n--- ЧАСТОТНЫЙ АНАЛИЗ ПОЛЬЗОВАТЕЛЬСКОГО ТЕКСТА -
--")
                    write_log(f"{'Буква':<6} {'Частота':<10}")
                    write_log("-" * 16)
                    for char, freq in stats:
                        line = f"{char:<6} {freq:.4f}"
                        write_log(line)

            elif crypto_sub == '3':
                ct = input("Введите шифр-текст для взлома: ").strip()
                if not ct:
                    print("Шифр-текст пуст.")
                else:
                    result = find_key_and_decrypt(ct)
                    if result:
                        a_found, b_found, text_found = result
                        write_log(f"\nИТОГ ДЛЯ РУЧНОГО ВВОДА:")
                        write_log(f"КЛЮЧ: a={a_found}, b={b_found}")
                        write_log(f"ТЕКСТ: {text_found}")
            else:
                print("Неверный выбор режима.")

        elif choice == '3':
            write_log("--- ВЫХОД ИЗ ПРОГРАММЫ ---")
            print("Выход. Все результаты сохранены в", LOG_FILE)
            break

        else:
```

```
            print("Пожалуйста, выберите 1, 2 или 3.")
run_menu()
```